Fine-Tuning LLMs with Multi-GPU Training in an HPC System

NRIS

Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info. Tech. Department, UiB/NRIS

Challenges with Fine-Tuning LLMs

Storage & Memory Bottlenecks

- GPT-3 (175B): checkpoint ≈ **350 GB (3.5TB for 10 tasks)**
- GPU memory requirement: **~1.2 TB**

What is Needed?

- **Optimization strategies** to overcome hardware limitations
- **High-Performance Computing (HPC)** with:
  - Powerful CPUs & GPUs
  - High-speed interconnects
- **Well-optimized software stack including e.g.**
  - Communication libraries
  - ML frameworks …etc

Why HPC Matters

- Without HPC systems, fine-tuning large LLMs (e.g., GPT-3 175B) on large datasets is nearly impossible.

Purpose of this Workshop

- Introduce **fine-tuning concepts**
- Provide **hands-on practice**
- Demonstrate **HPC-specific execution**

Workshop Program

09:30 – 10:15 | Introduction to Fine-Tuning & Optimization Strategies (30+15min)

- Overview of LLM and Fine-tuning
- Concepts: Parameter-efficient methods (LoRA, LoRA Dropout)
- Model merging

10:30 – 11:30 | Environment Setup & Configuration Overview (Olivia)

- Overview of Olivia Supercomputer
- Setting up training environment - **interactive session**
- Config file deep dive of Llama

Hands-On Session – Practical Fine-Tuning & Inference

Task 1: Question-Answering (Alpaca dataset)

12:30 – 13:30 | Fine-tuning on 1 GPU + Inference (60 min)
13:45 – 14:00 | Profiling (15 min)
14:00 – 14:45 | Fine-tuning on multiple GPUs + Inference (35 min)

Task 2: Choose your own experiment — open-ended experiment

15:00 – 15:15 | Wrap-up & Discussion (for early leavers)
15:15 – 16:00 | Hands-on continuity

- Try e.g. summarization (Xsum dataset) with LoRA vs. full fine-tuning.
- Compare single vs. multi-GPU scaling.
- Adjust config parameters (learning rate, batch size, epochs) and compare outputs.

Learning Outcomes

- Learn about concepts of LoRA and LoRA Dropout
- Apply LoRA fine-tuning to a LLaMA model.
- Set up PyTorch+extra packages using singularity on an HPC system
- Get familiar with training configuration files for LlaMA models.
- Run training jobs on a single GPU and scale up to multiple GPUs.
- Perform inference tasks such as QA and summarization.
- Monitor GPU usage and memory utilization & Profiling

Introduction to Fine-Tuning & Optimization Strategies

- Overview of LLM and Fine-tuning
- Concepts: Parameter-efficient methods (LoRA, LoRA Dropout)
- Model merging

Overview of Large Language Models (LLMs)

- **LLM** is a type of AI model trained on vast amounts of **text data** to understand and generate **human-like language**.

- **LLM scope** includes tasks such as:
  - Text generation
  - **Text summarization**
  - **Question Answering (QA)**
  - Translation
  - Code generation

- **Fine-tuning** is the process of**:**
  - Adapting a pre-trained model (like an LLM)
  - Using **task-specific or domain-specific data**
  - Continuing training on a smaller, targeted dataset
  - Updating all the parameters of the pre-trained model.

- **Why Fine-Tuning ?** It enables:
  - Faster training (without re-training the full model)
  - Better performance on domain-specific tasks (or specific datasets)
  - Reduced data requirements

Pre-training vs Fine-tuning LLM

Aspect	Pre-training	Fine-tuning
Definition	Training on a vast amount of unlabelled text data	Adapting a pre-trained model to specific tasks
Data Requirement	Extensive and diverse unlabelled text data	Smaller, task-specific labelled data
Objective	Build general linguistic knowledge	Specialise model for specific tasks
Process	Data collection, training on large dataset, predict next word/sequence	Task-specific data collection, modify last layer for task, train on new dataset, generate output based on tasks
Model Modification	Entire model trained	Last layers adapted for new task
Computational Cost	High (large dataset, complex model)	Lower (smaller dataset, fine-tuning layers)
Training Duration	Weeks to months	Days to weeks
Purpose	General language understanding	Task-specific performance improvement
Examples	GPT, LLaMA 3	Fine-tuning LLaMA 3 for summarisation

Ultimate Guide to Fine-Tuning LLMs https://arxiv.org/pdf/2408.13296

Challenges of Full Fine-tuning LLM

Full fine-tuning (updating all parameters for each task) becomes expensive.

- **High GPU Memory:** Requires immense GPU memory

  (e.g., **1.2TB for GPT-3 175B –** trainable parameters)

- **High Storage Cost:** Storing instances of fine-tuned models for each task requires **massive storage**

  (e.g., 10 fine-tuned GPT-3 models would be **350GB (size of checkpoint)x100= 3.5TB**).

- **High computational cost for training:** Require significant compute resources from days to weeks depending on the model size and dataset scale.

Need of optimizing fine-tuning LLM

Introducing LoRA (Low-Rank Adaptation)

LoRA https://arxiv.org/pdf/2106.09685

LoRA Overview

- **LoRA** is an optimization technique to to reduce **compute and storage costs** of full fine-tuning.

(e.g., LoRA cuts compute by 10x)

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

- **Core Idea:** Freeze the pre-trained model weights (the main model's weights do not change during fine-tuning)
  Inject trainable rank decomposition matrices into each layer of the Transformer architecture.

- **Concept:** Only **train low rank** matrices **A & B**, while keeping the **pre-trained weight matrix frozen**.

Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + BAx.$$

$\Delta W = BA$ Low rank decomposition

LoRA https://arxiv.org/pdf/2106.09685

$W_0 \in \mathbb{R}^{n_1 \times n_2}$ Pre-trained weight matrix (frozen)

$A \in \mathbb{R}^{r \times n_2}$

$B \in \mathbb{R}^{n_1 \times r}$

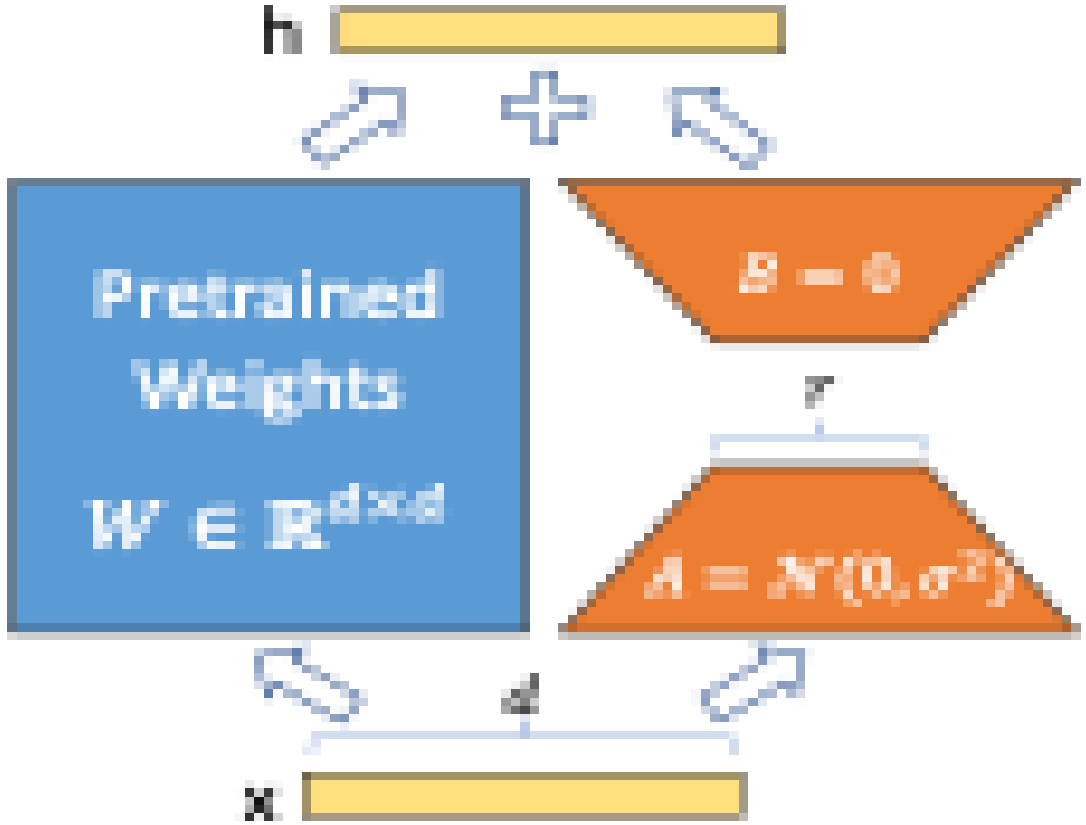
Low-rank update (trainable)

x Input vector

h Output of the forward pass

LoRA's Efficiency: example

- **Full Fine-tuning:** A pre-trained weight matrix **W0** with a dimension of **800x800 has 640,000** trainable parameters.
- **LoRA Fine-tuning:** With a **rank of 8**, the number of trainable parameters is reduced to 8x800(**A**) + 800x8(**B**)=**12,800**.
- **Factor of Reduction:** The number of trainable parameters is reduced by a factor of **50 times** (640,000 / 12,800 = 50).
- **Benefit:** This significant reduction lowers both computational and storage costs.



LoRA https://arxiv.org/pdf/2106.09685

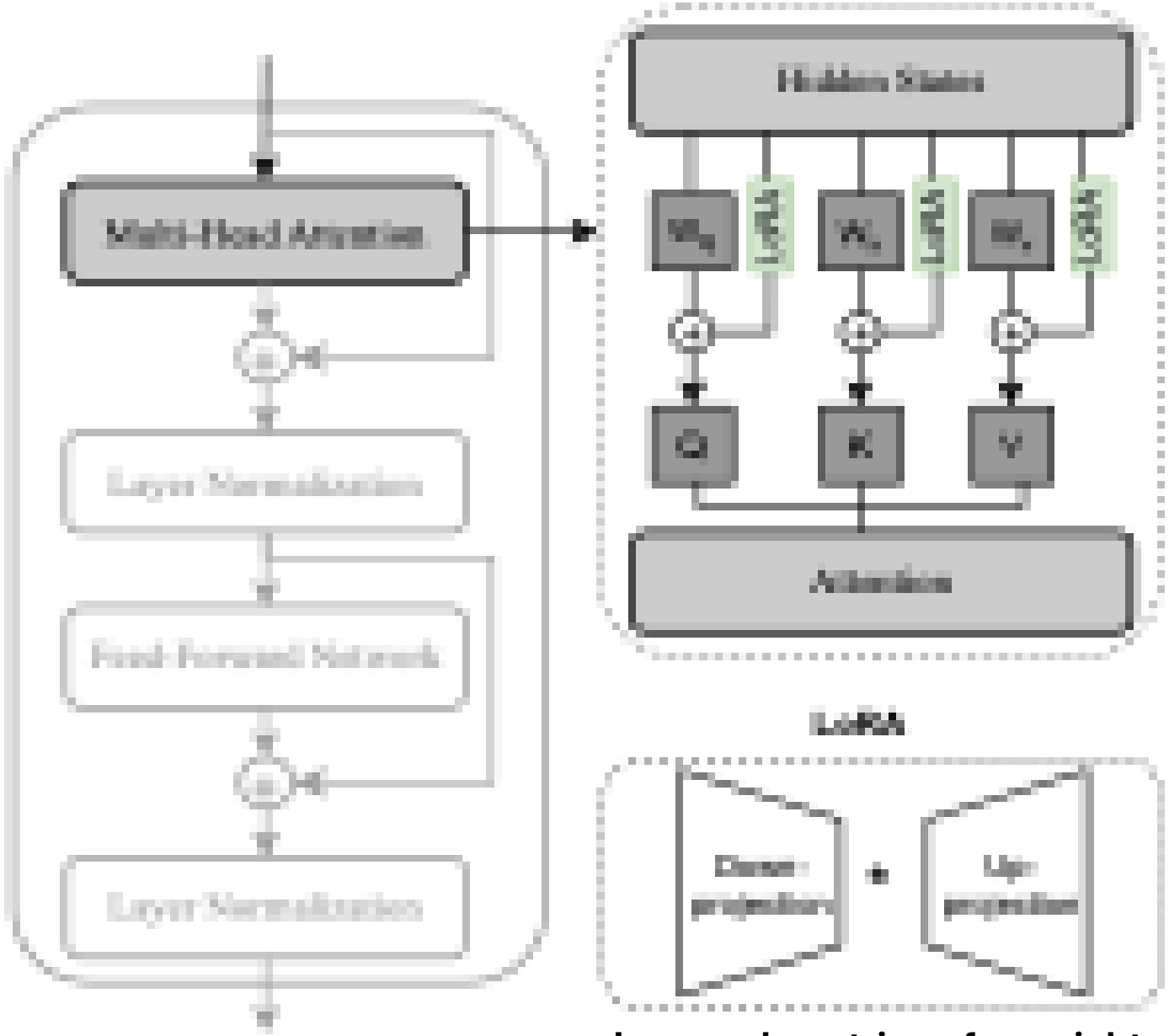
Where LoRA happens ?

- **In a Transformer, Each attention head**: transforms (linear) the **input embeddings** into: **Query (Q), Key (K)** and **Value (V)** vectors.

  - **"Query"** vector → what information a token is looking for.
  - **"Key"** vector → what information the token contains.
  - **"Value"** vector → contains the actual content to be aggregated.

Fastformer https://arxiv.org/pdf/2312.12148

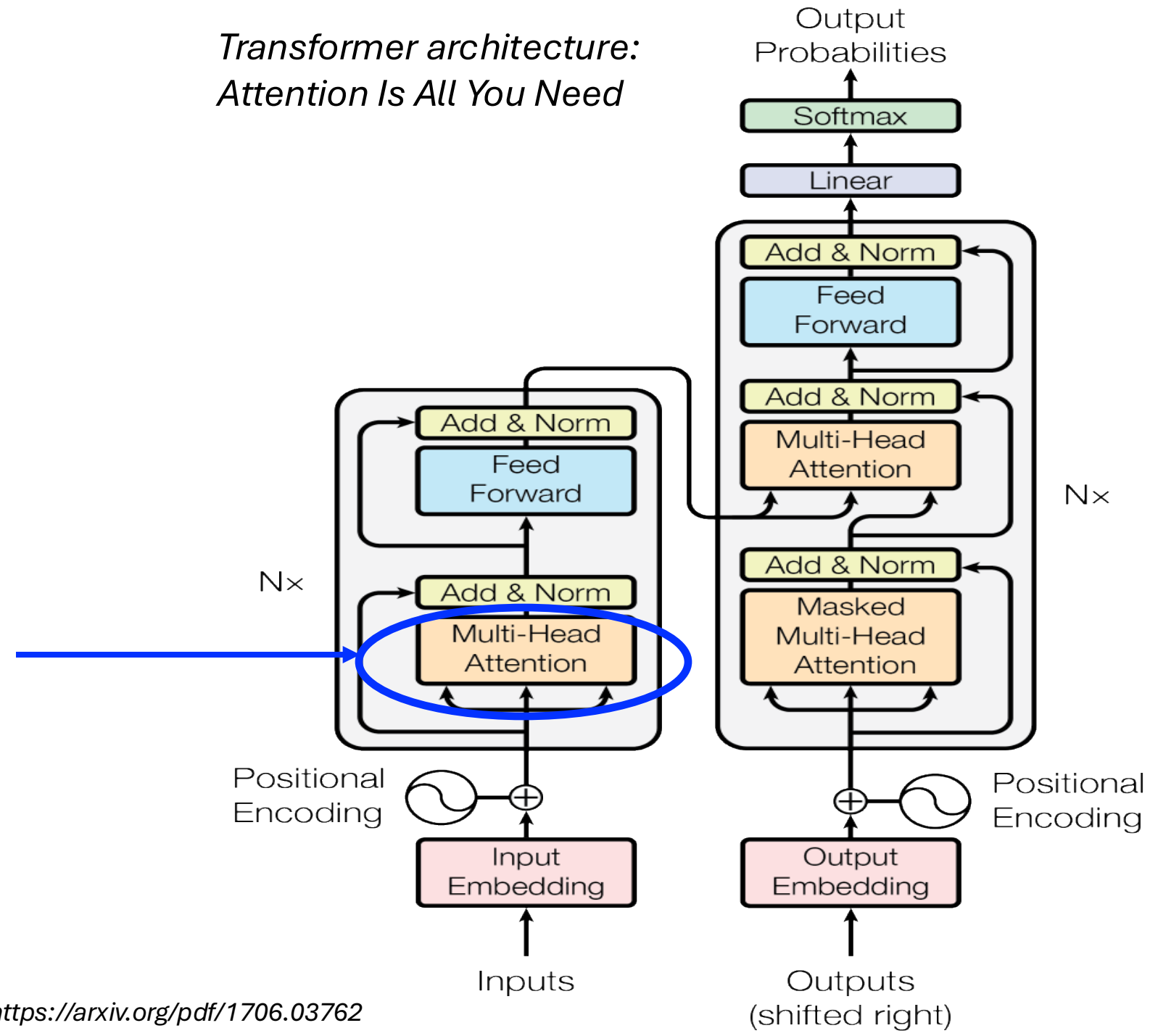
Architecture of a **Transformer** block



PEFT https://arxiv.org/pdf/2312.12148

Architecture of Transformer

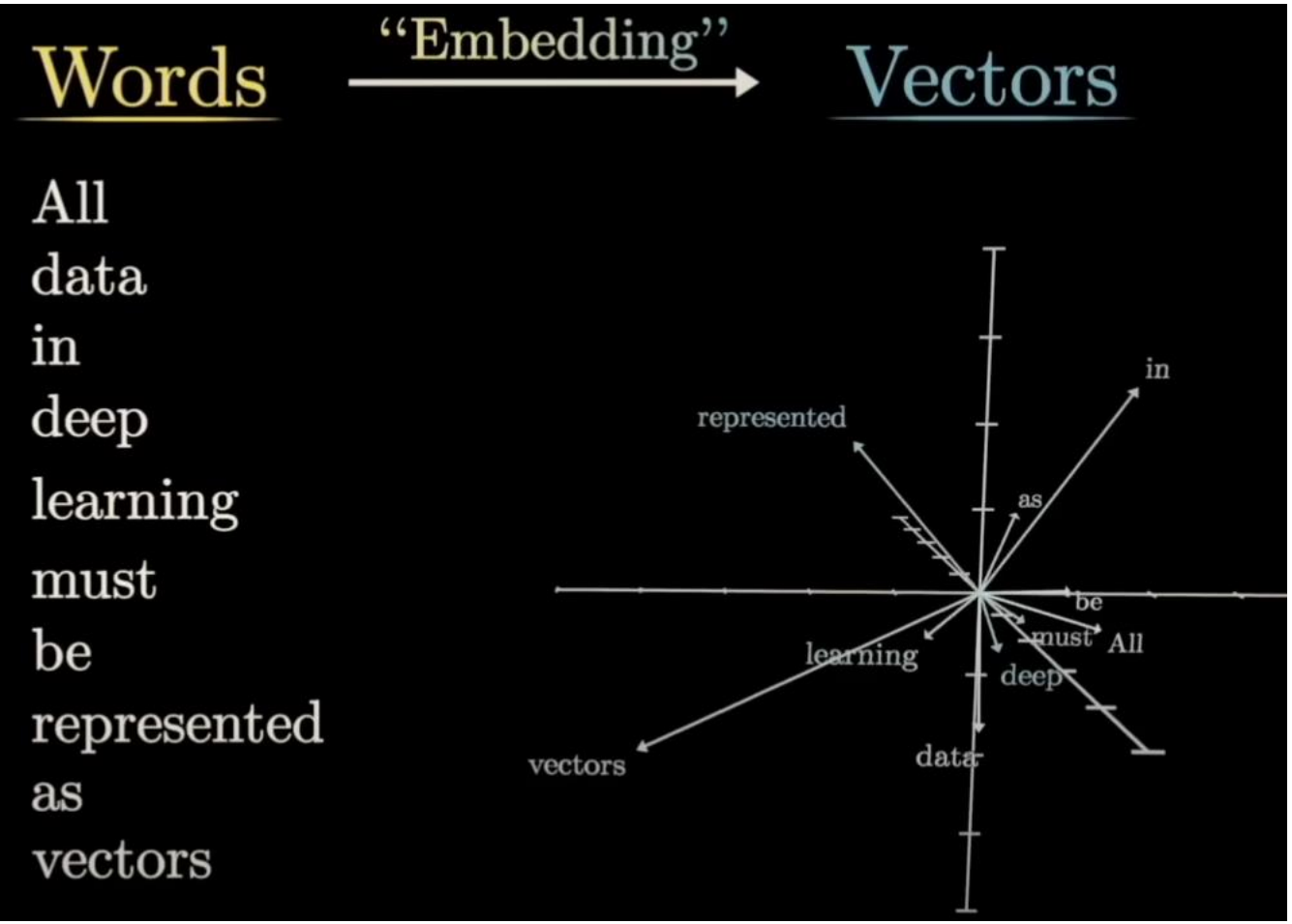
Transformer architecture: Attention Is All You Need

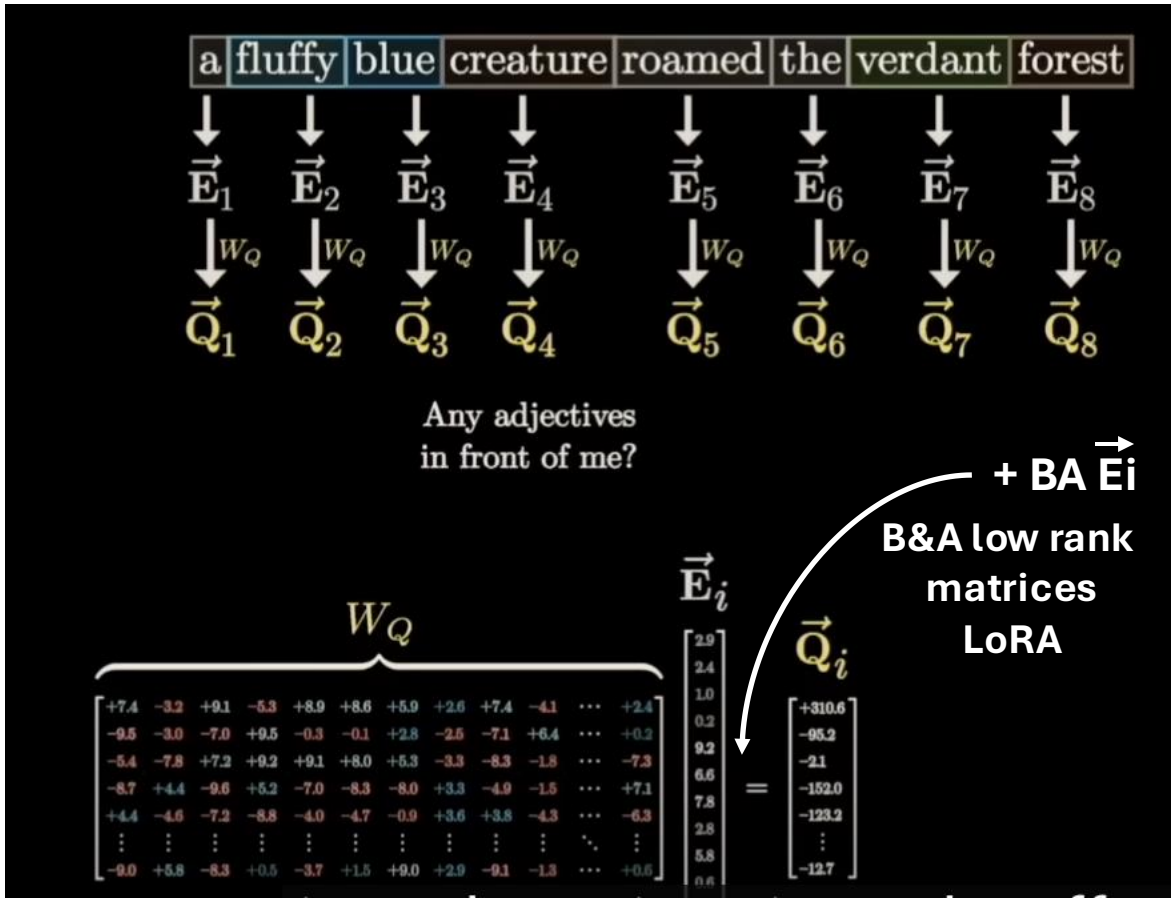


https://arxiv.org/pdf/1706.03762

Word Representations in Vector Space

Associating words with vectors (embedding)



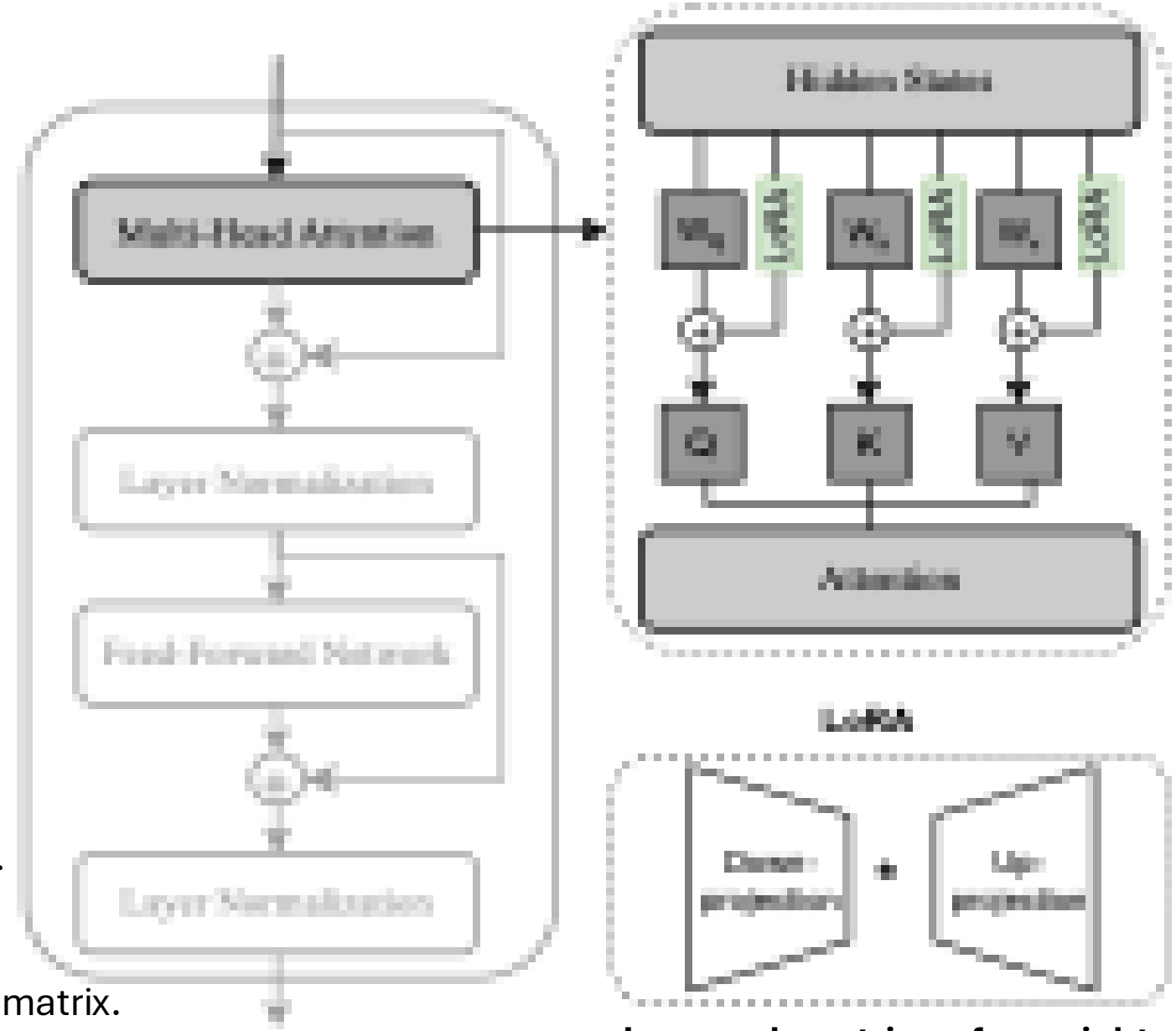


- **Ei** = word embedding for token i.
- **WQ** = learned weight matrix that converts embeddings into queries.
- **Qi** = query vector, representing what the word wants to attend to.

Visualizing transformers and attention | Talk for TNG Big Tech Day '24
https://www.youtube.com/watch?v=KJtZARuO3JY&t=155s
Efficient Estimation of Word Representations in Vector Space
https://arxiv.org/pdf/1301.3781

Where LoRA happens ?

Architecture of
a **Transformer** block



- **In a Transformer, Each attention head**: transforms (linear) the **input embeddings** into: **Query (Q), Key (K)** and **Value (V)** vectors.

- **LoRA** is implemented inside each **Head Attention.**

- The **LoRA matrices** are **injected into the attention layer's weight matrices**, specifically, **Wq and Wv projection matrices**.

$$W'_Q = W_Q + \Delta W_Q, \quad \Delta W_Q = B_Q A_Q$$

$$W'_V = W_V + \Delta W_V, \quad \Delta W_V = B_V A_V$$

Wq transforms the embedding vectors into **Query vectors**.

- In **LoRA**, **Wq, Wk, Wv, are frozen** pre-trained weight matrix.

Low-rank matrices for weights

PEFT https://arxiv.org/pdf/2312.12148

Merging LoRA weights into the base model

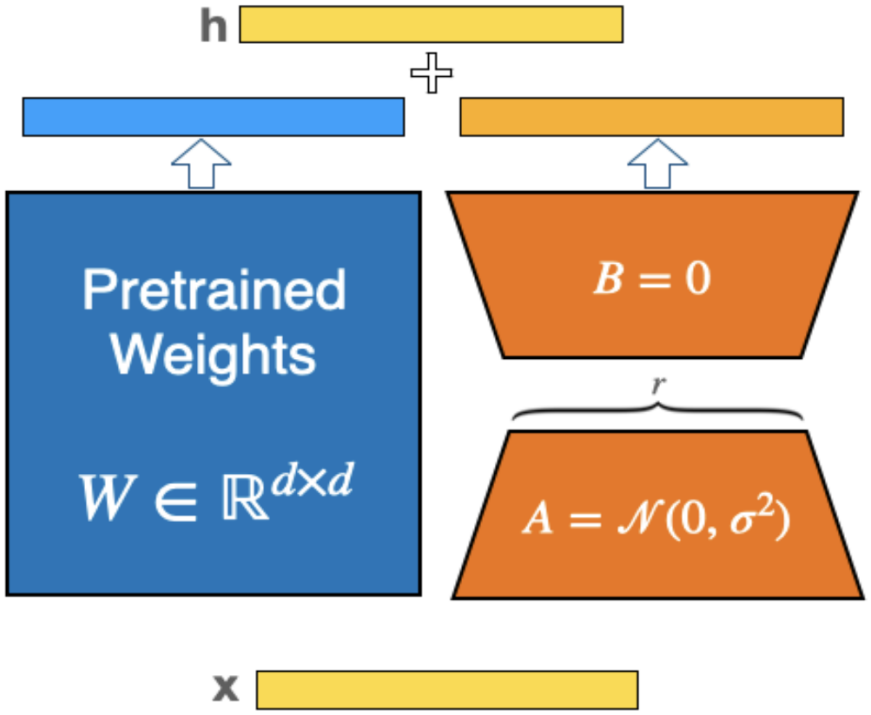
Key Benefits of LoRA

- **Direct merging:** Trained low-rank matrices (A & B) can be merged into the original pre-trained weights (W).
- **No added latency:** Merging does not slow down inference.
- **Efficient storage:**
  - Avoids saving multiple large checkpoints.
  - Example: 10 tasks × 350 GB each = 3.5 TB → only a few MB for LoRA weights.
- **Flexible deployment:** LoRA weights can be merged into the base model for inference.
- **Merging options:**
  - At the end of fine-tuning (with pre-trained weights).
  - During inference as needed.

Merging LoRA weights into the base model

- **Model merging** can be done either after the training or during the inference.

During the training



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

After training

Merged Weights

$W_{merged} \in \mathbb{R}^{d \times d}$

8B model (each element is **4 bytes** (fp32)) -→ **32GB**
10 tasks ---→ 320 GB of stoarge
Total number of parameters saved **is dxd**

LoRA Weights

$A \in \mathbb{R}^{r \times d}$
$B \in \mathbb{R}^{d \times r}$

10 tasks ---→ 32 + 0.5 (LoRA weight) GB of stoarge
Total number of parameters saved **is 2dxr**

https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora

Advantages of LoRA

- **Massive Parameter Reduction:** Reduces trainable parameters significantly
  (e.g., **10,000 times reduction** for GPT-3 175B compared to fine-tuning).
- **Reduced GPU Memory:** Lowers GPU memory requirements
  (e.g., **3 times less VRAM** for GPT-3 175B, from 1.2TB to 350GB).
- **Reduced storage cost:** e.g. GPT-3 175B **reduce checkpoint size from 350 GB to 35 MB** for LoRA weights.
- **Improved Training Throughput:** Faster training due to fewer parameters needing gradient calculation and optimizer states.
  (e.g., **25% speedup** on GPT-3 175B).
- **Efficient Task Switching:** **Rapidly switch between tasks** by simply swapping the small LoRA matrices,
  instead of reloading the entire model.
- **No Inference Latency:** Trainable LoRA matrices (A and B) can be merged with the frozen pre-trained weights.

LoRA https://arxiv.org/pdf/2106.09685

Why LoRA seems to work ?

- **Low intrinsic rank hypothesis**: Actual update required to adapt to a new task lies in a much smaller subspace.
  No need for updating the pre-trained weights.
  Key information is captured even with low ranks.

- **Empirical experiments**: Tests show that even very small LoRA ranks can match full fine-tuning performance.

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
[redacted]				
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

LoRA https://arxiv.org/pdf/2106.09685

Risk of fine-tuning using LoRA

LoRA requires optimization of low rank for optimal performance.

→ **Risk of overfitting on the training dataset**
(affects the effectiveness of the fine-tuned model)

LoRA Dropout is a mechanism designed to overcome overfitting in LoRA-based methods

---

LoRA Dropout as a Sparsity Regularizer for Overfitting Control

---

Yang Lin[1 2*] Xinyu Ma[1 2*] Xu Chu[1 2] Yujie Jin[1 2] Zhibang Yang[2] Yasha Wang[1 2] Hong Mei[1]

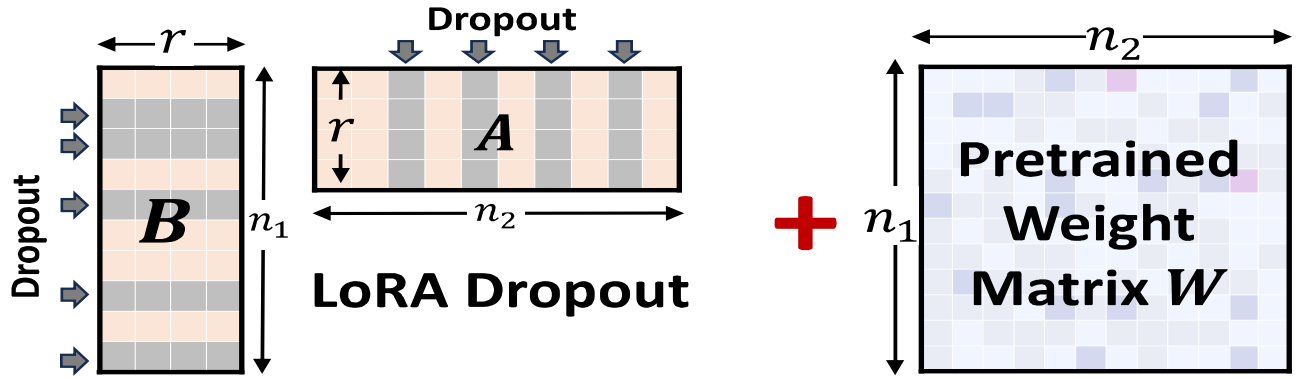
LoRA Dropout as a Sparsity Regularizer for Overfitting Control, https://arxiv.org/pdf/2404.09610

Concept of LoRA Dropout – for Overfitting control

Randomly drop rows and columns from both tunable low-rank parameter matrices **A & B**:

$$\hat{A} = A \cdot \mathrm{diag}(m_A), m_A \sim \mathrm{Bern}(1-p);$$

$$\hat{B} = \left(B^\top \cdot \mathrm{diag}(m_B)\right)^\top, m_B \sim \mathrm{Bern}(1-p),$$



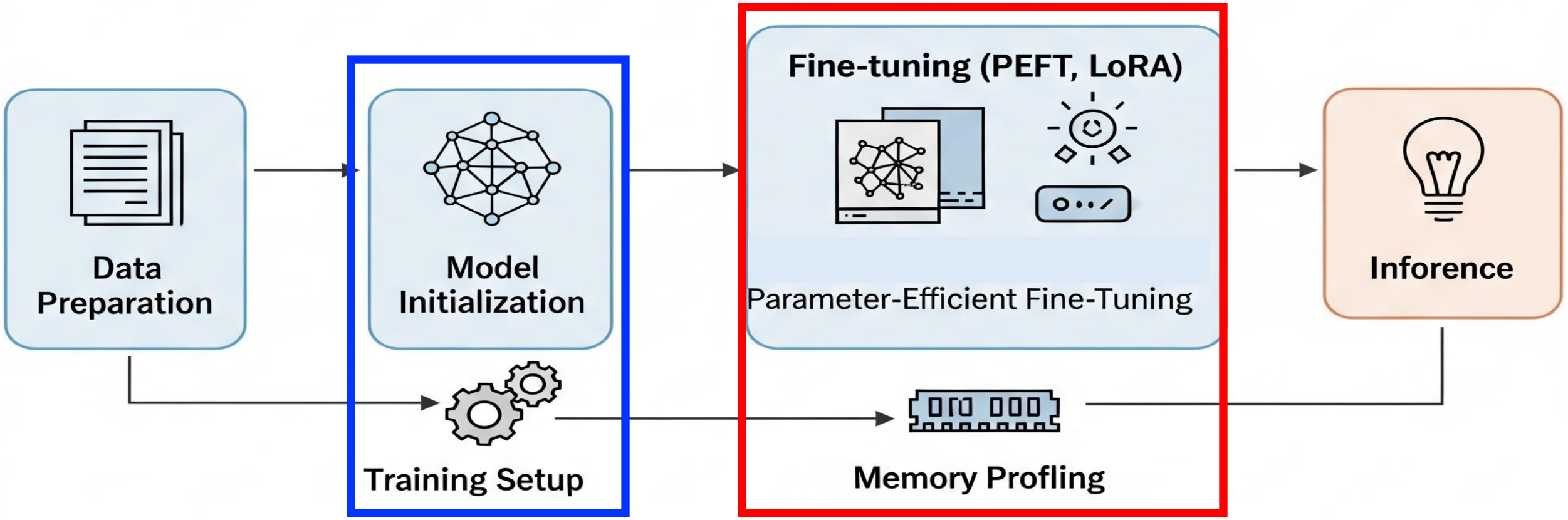
Drop column in A: Zero out a column → that dimension is ignored

- **p=0**, no dropout, which means the matrices A and B are unchanged. $A \cdot \mathrm{diag}(m_A) = A \cdot I = A$
- **p=1,** "dropping" all the information contained in **A and B**. (**A & B are zeros** - **No LoRA training**)
- **p=0.1,** means that 10% of the rows in matrix B and 10% of the columns in matrix A will be randomly dropped (set to zero) during each training step.

Method	#Params	MNLI M-Acc	SST-2 Acc	CoLA Mcc	QQP Acc	QNLI Acc	RTE Acc	MRPC Acc	STS-B Corr	All Avg.
Full Fine-Tuning	184M	89.90	95.63	69.19	**92.40**	94.03	83.75	89.46	91.60	88.25
LoRA$_{r=8}$	1.33M	90.65	94.95	69.82	91.99	93.87	85.20	89.95	91.60	88.50
LoRA+Dropout	1.33M	**90.85**	95.87	71.32	92.22	**94.56**	88.09	**91.42**	92.00	89.54

LoRA Dropout as a Sparsity Regularizer for Overfitting Control, https://arxiv.org/pdf/2404.09610

Fine-tuning pipeline for LLM

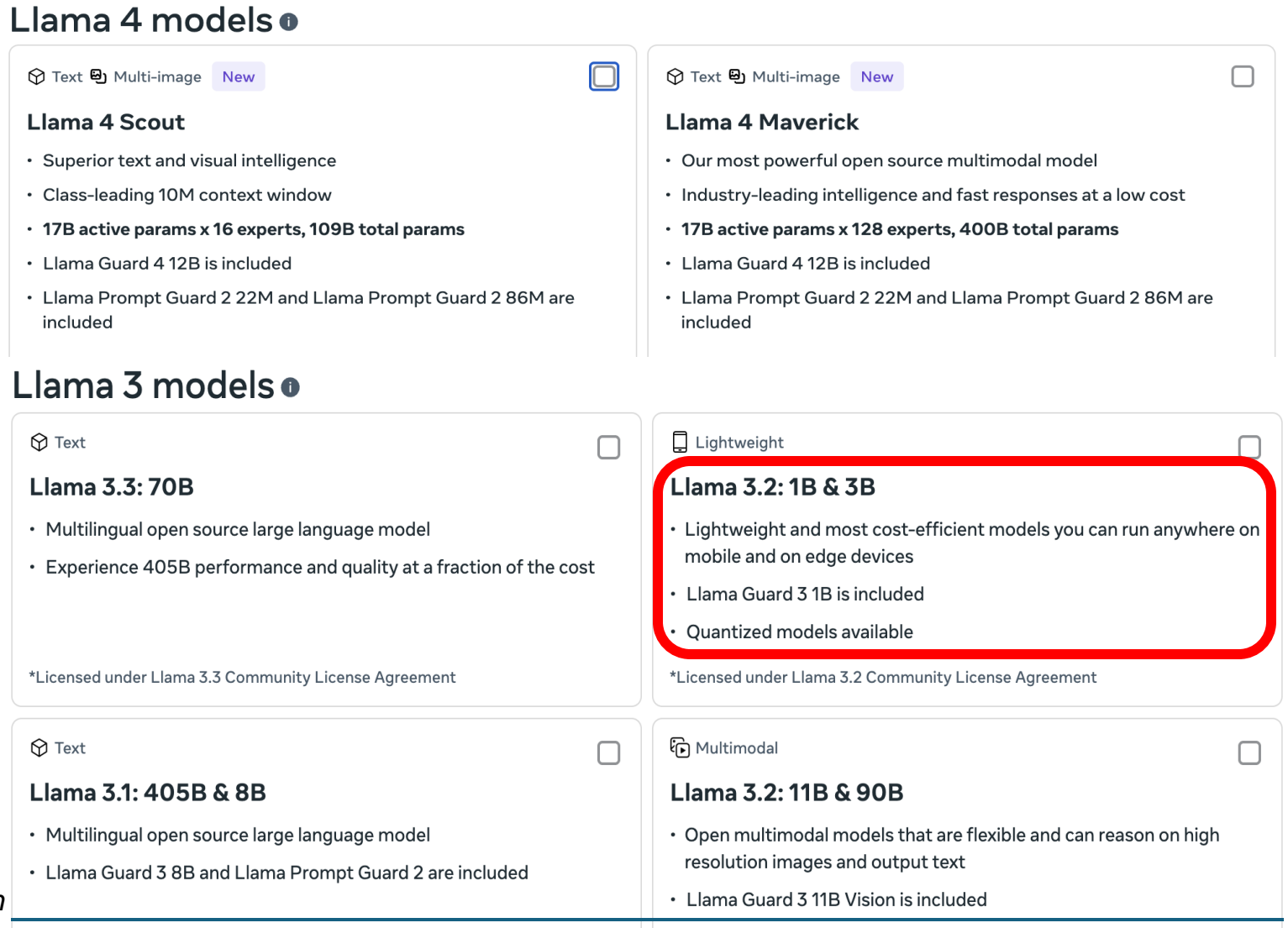


State-of-the-art LLaMA

Modern LLMs are primarily based on the **Transformer** architecture (see *https://arxiv.org/pdf/1706.03762*)

LLaMA developed by **Meta AI** as an open-source alternative to proprietary LLMs e.g. GPT

State-of-the-art LLaMA:



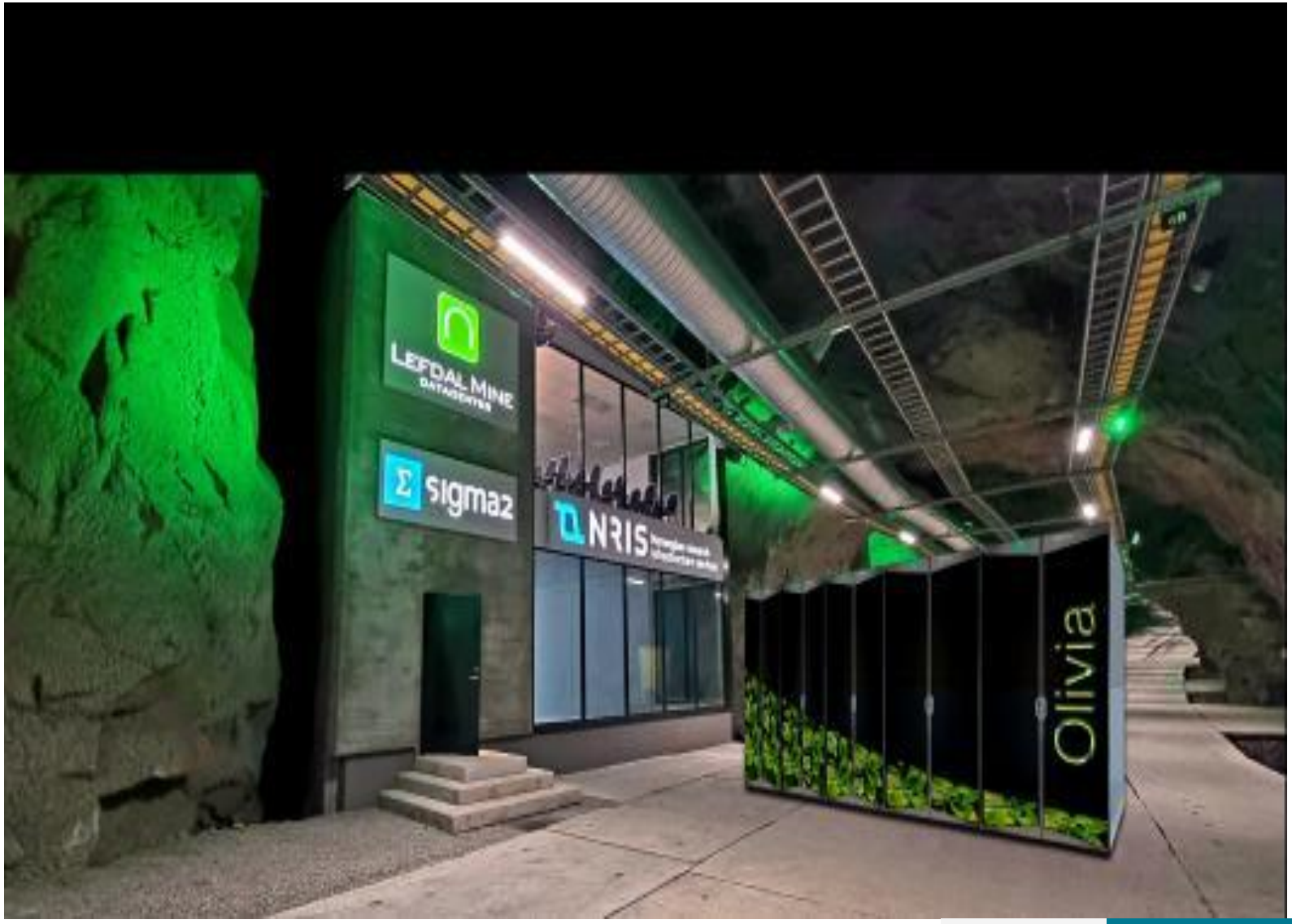
www.llama.com

Environment Setup on Olivia Supercomputer

- **Overview of Olivia supercomputer**
- **SSH setup on Olivia**
- Setting up **PyTorch**, **Torchao**, **Torchtune & peft**
- Verifying **CUDA support** and GPU availability

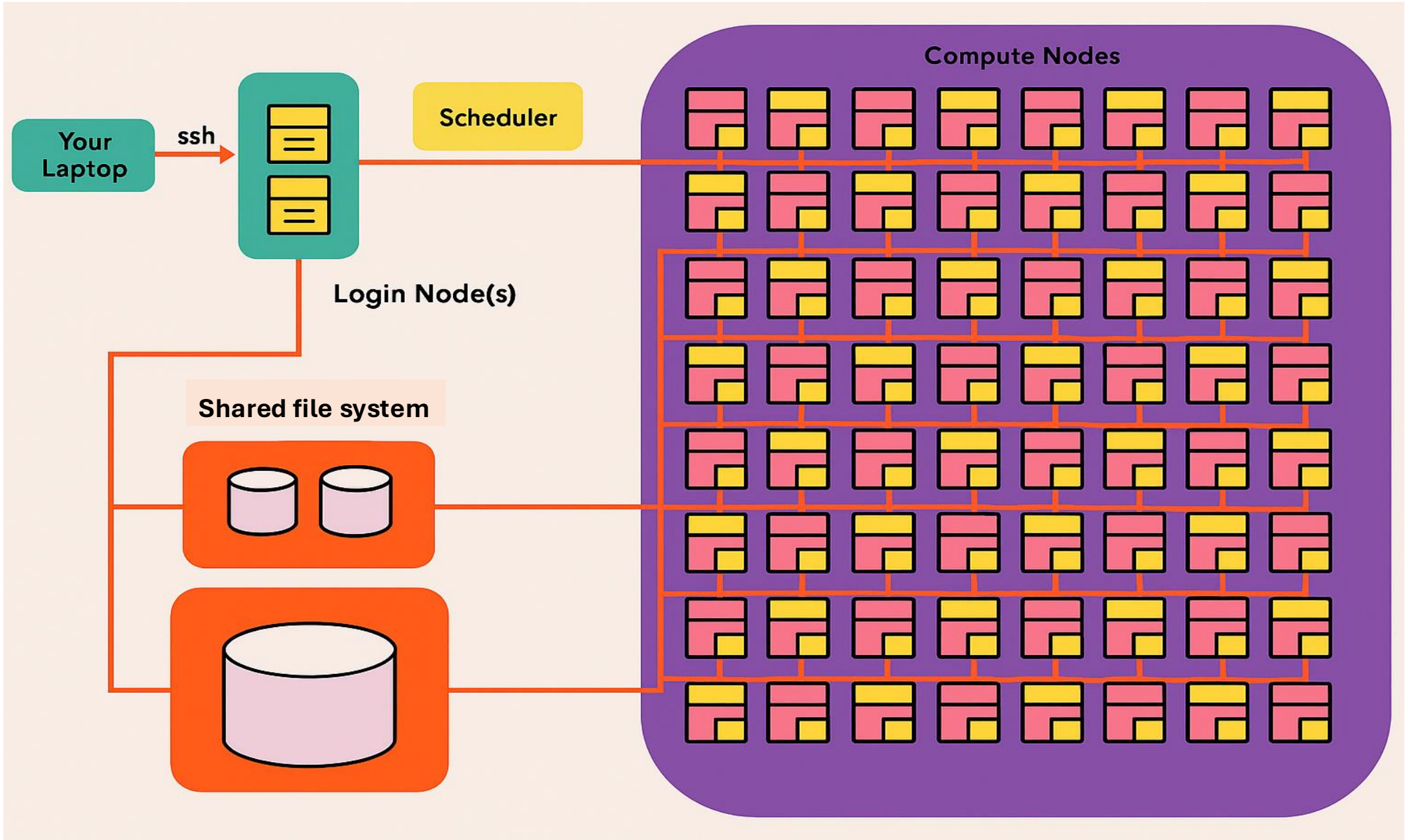
Norway's New Supercomputer - Olivia

- **Olivia** is located in Lefdal Mine (Kjølsdalen)
- System: **HPE Cray** Supercomputing EX
- The GPU partition consists of:
  - 76 GPU-nodes (**304** GPUs)
  - NVIDIA **Grace Hopper Superchip** (**GH200** 96 GB)
  - **HPE Slingshot Interconnect**



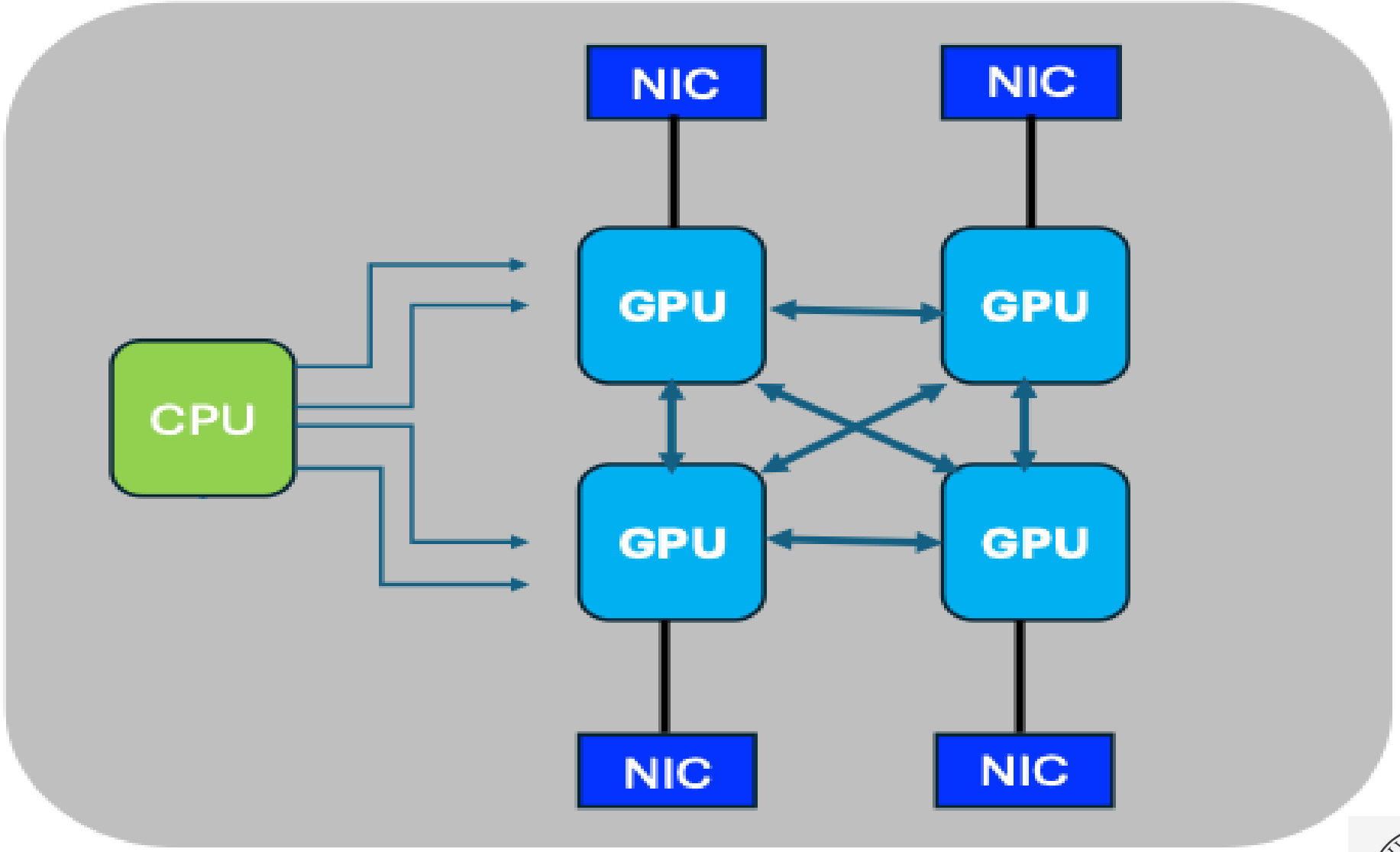
https://www.sigma2.no/news/2024/olivia-becomes-name-norways-next-supercomputer

Basic Cluster-Architecture



Adapted from https://indico.cern.ch/event/814979/contributions/3401193/attachments/1831477/3105158/comp_arch_codas_2019.pdf

Compute node diagram (Simplified)

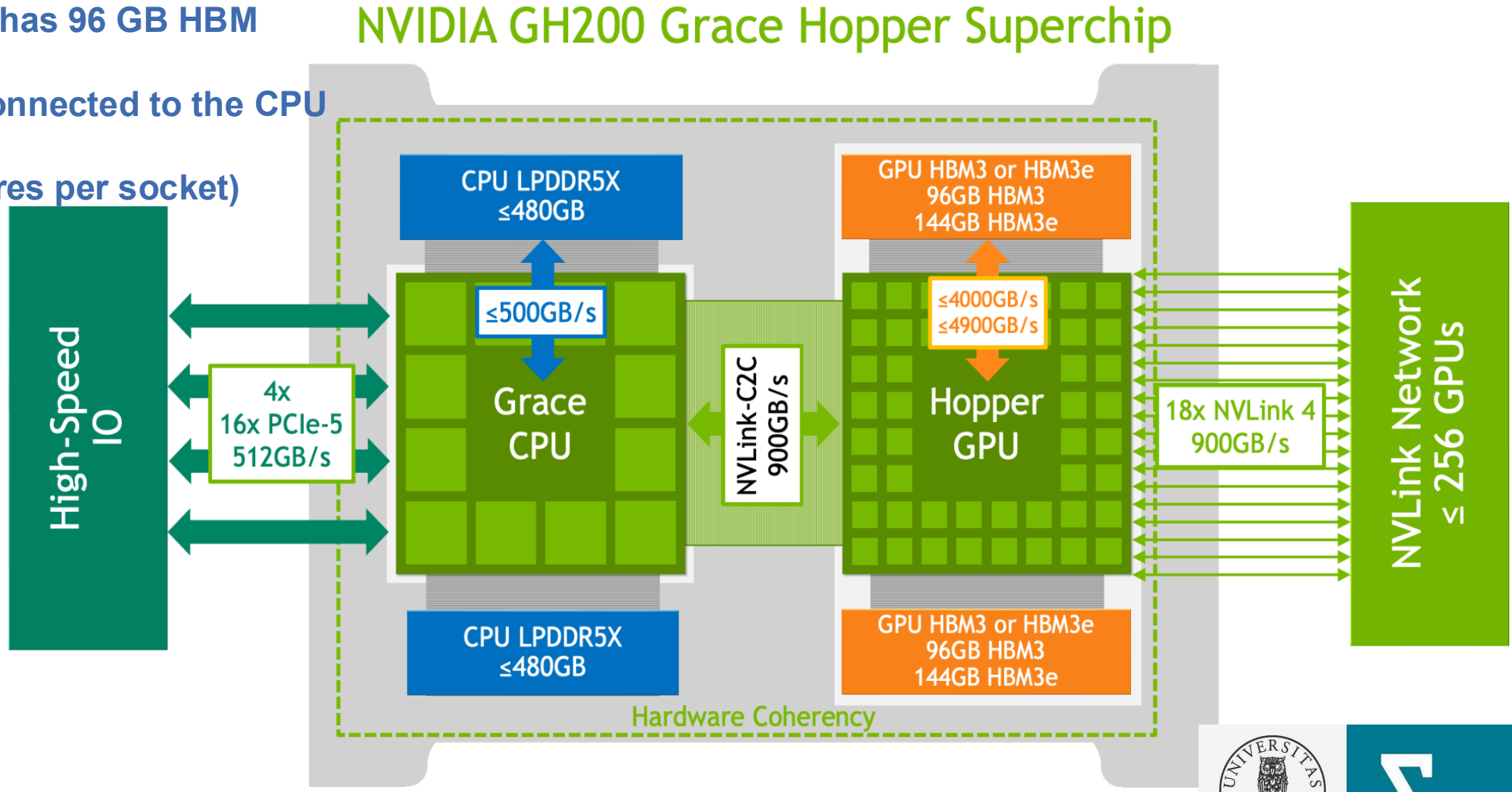


Overview of the Grace Hopper System

Olivia:
Each GH200 device has 96 GB HBM

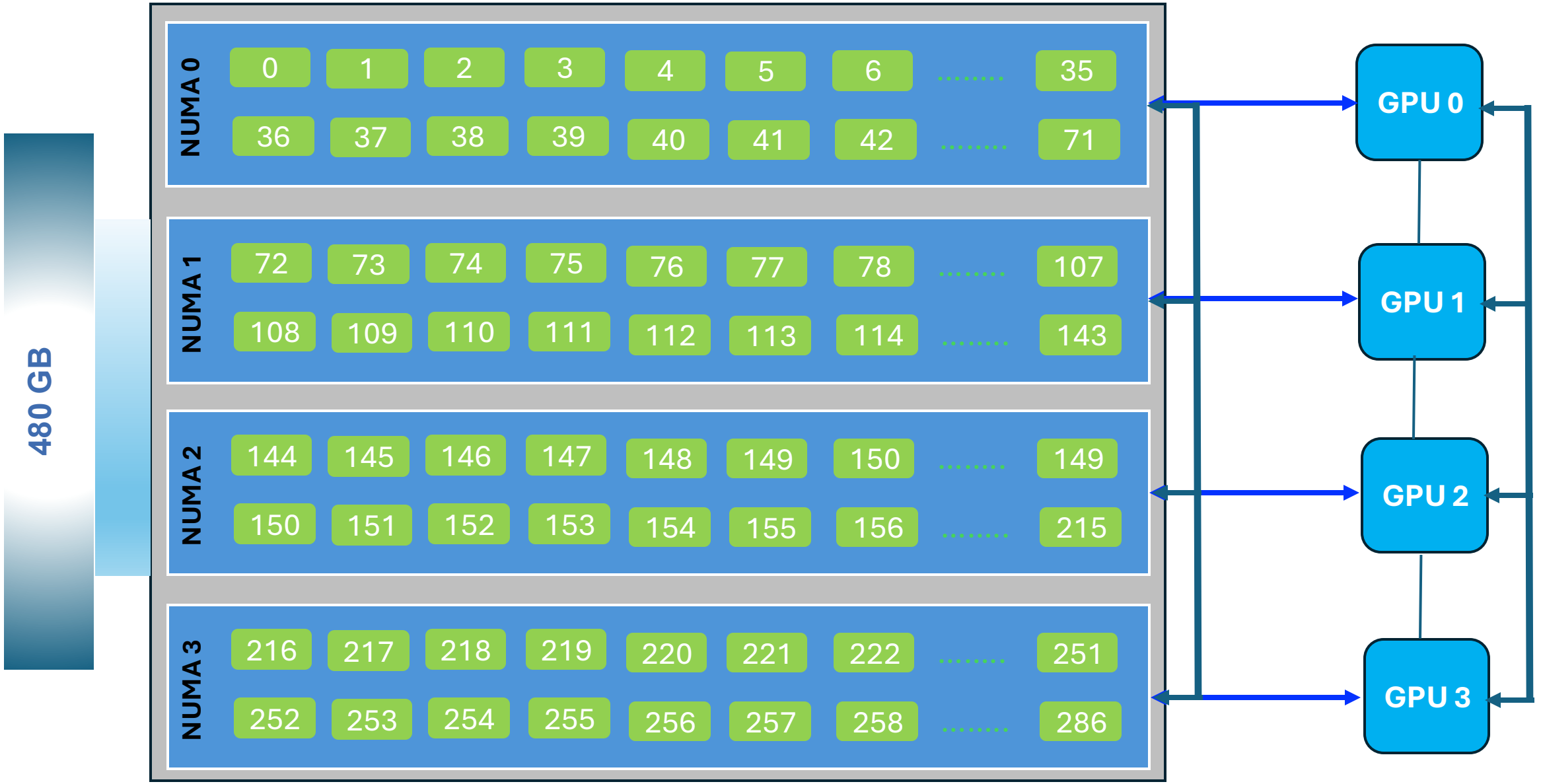
120 GB LPDDR5x connected to the CPU

288 cpu-core (72 cores per socket)



https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper

CPU-to-GPU affinity - (simplified diagram)



CPU-to-GPU affinity: Olivia system

- Binding a process or thread to a specific GPU.
- Workload is consistently executed on the specified GPU(s).
- Improve performance by reducing data transfer overhead.

```
hicham@x1000c0s1b1n0:~> nvidia-smi topo --matrix
        GPU0    GPU1    GPU2    GPU3    CPU Affinity    NUMA Affinity   GPU NUMA ID
GPU0     X      NV6     NV6     NV6     0-71    0               4
GPU1    NV6      X      NV6     NV6     72-143  1               12
GPU2    NV6     NV6      X      NV6     144-215 2               20
GPU3    NV6     NV6     NV6      X      216-287 3               28
```

GPU	CPU Affinity	NUMA Affinity	GPU NUMA ID
GPU0	0–71	0	4
GPU1	72–143	1	12
GPU2	144–215	2	20
GPU3	216–287	3	28

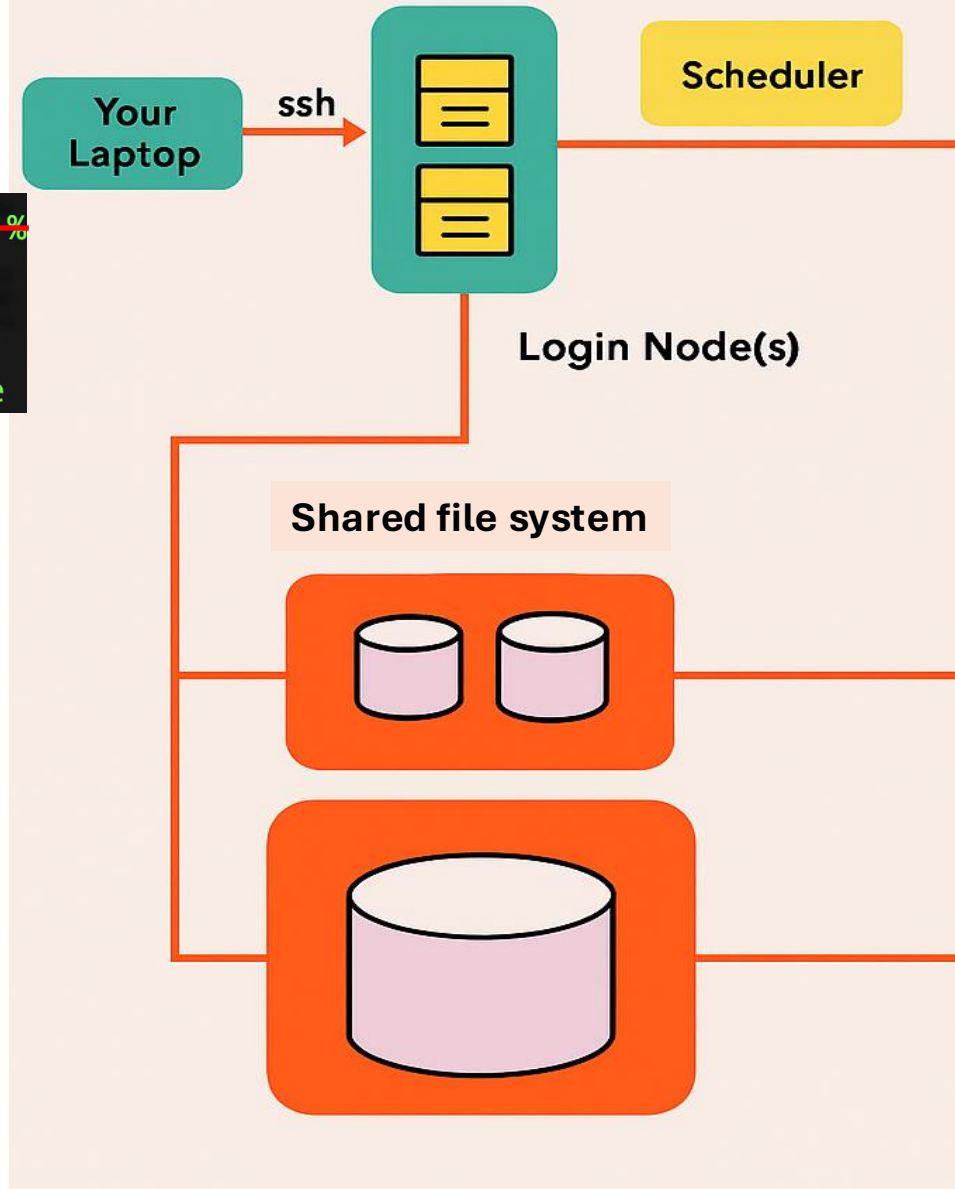
--cpu-bind=map_cpu:... → explicitly binds each task to the CPU cores nearest its GPU.

SLURM will then match **rank 0 → GPU0 (cores 0–71)**, **rank 1 → GPU1 (cores 72–143)**, ... & so on.

File System Storage

Divide storage into several filesystems: **/cluster; /scratch; /flash**
Spaces can be physically or logically separated

```
hicham@uan02:~> df -H -t lustre | awk '{sub(/.*:/,"",$1); printf "%-25s %8s %
Filesystem                Size     Used    Avail  Use% Mounted
/cluster                  3.2P      39T     3.2P    2% /cluster
/scratch                  1.2P     128T     1.1P   12% /cluster/work
/flash                    524T     384G     519T    1% /cluster/software
```



Persistent Project Storage: /cluster

- **Purpose:** Shared, long-term files for your entire project lifecycle.
- **Policy:** Data is **meant to stay** until the project ends.

High-Speed Scratch: /scratch

- **Purpose: Temporary workspace** for active jobs (high-throughput I/O).
- **Policy:** Files are **not backed up** and are **automatically deleted** after a variable time (e.g., 30–90 days).

- **Warning:** Expect files to be purged sooner if the system is full. **Do not store critical files here.**

End PART 1